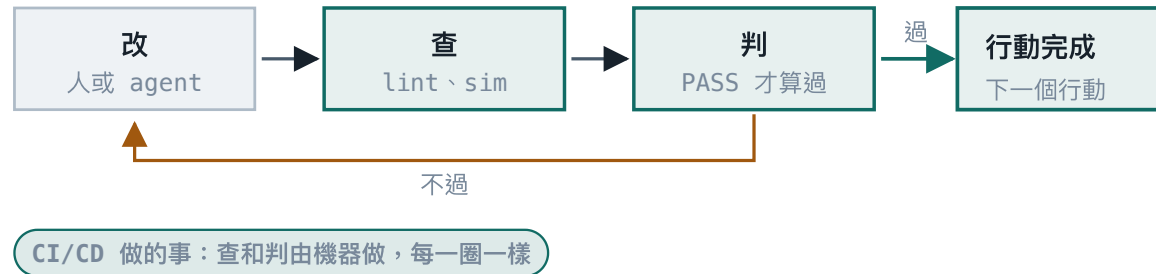


迴圈：一個行動是改、查、判轉到過為止，CI/CD 把查和判交給機器

這份文件回答：CI/CD 對這個團隊的意義是什麼；《把版控當備份的團隊》裡的症狀，代價到底多大 ↓

一個行動 = 改、查、判，轉到過為止



一個行動的例子（示意）

行動	給 DMA 加 burst 模式
改	改 RTL，加一個 testcase
查	lint、compile、跑 DMA 的 sim 子集
判	全部 PASS 才過；有一個 FAIL 就回去改

用語：軟體業把 commit 前叫 inner loop（本機的改、lint、unit test），commit 後的整合與 CI/CD 叫 outer loop。

這份文件的 inner loop 指「一個行動的迴圈」，outer loop 指後面會講的「探索」：比較 N 個候選。

CI/CD 把查和判交給機器：每一圈一樣的環境、一樣的標準、不用人在旁邊。

人或 agent 只負責改；迴圈什麼時候轉、轉幾圈，都和人的空檔無關。

半徑：IC 的一圈有快有慢，CI/CD 把它們分層排程，結果由機器判

一圈要多久

長條是示意，不代表任何實際時間

CI/CD 怎麼排

lint、compile

快



每次改完就跑

人或 agent 等得起

sanity sim (子集)

中

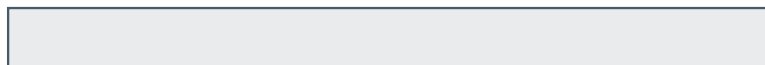


每包 submit 都跑

排進 queue，結果回來機器判

完整 regression

慢

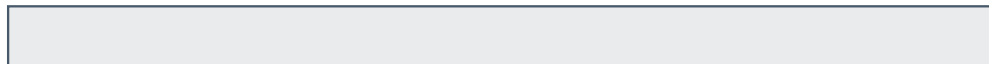


每晚跑一次

早上看結果，附 CL 號

synthesis、STA

很慢



每個里程碑、每個候選各跑一次

跑完自動抓 PPA

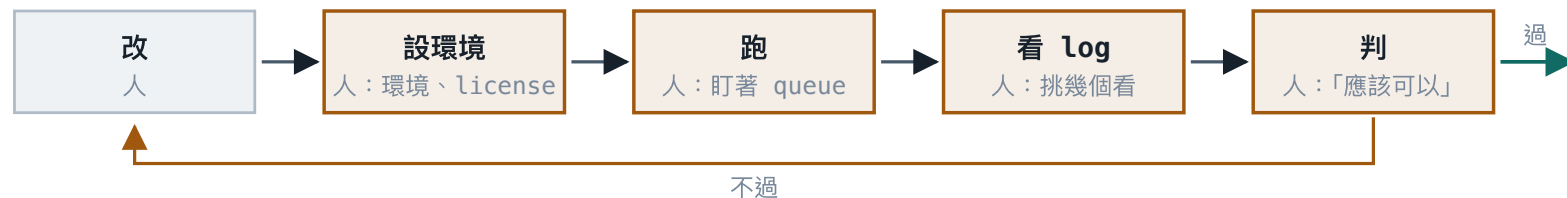
快的每圈都跑、慢的定時跑；哪一層都由機器判，結果放在大家看得到的地方。

CI/CD 不會讓 sim 或 synthesis 變快；它做的是讓每一層不用人顧、結果能信，慢的那幾層在背景轉。

agent 能自己轉的是快的那幾層；慢的那幾層由它排進去、等結果。

沒有 CI/CD 的 inner loop：每圈都要人設環境、人跑、人判，轉得慢又判不準

同一個行動，沒有 CI/CD 的時候



每一圈都卡在人身上；agent 改完也一樣，要等人設環境、人跑、人判。

每一圈的代價

- ✗ 一圈的時間 = 那個人的空檔
- ✗ 兩圈的環境不保證一樣，比不出好壞
- ✗ 判的標準每個人不同
- ✗ 結果對不回是哪一圈改的

圖 1

agent 的處境

圖 14

- 改得很快，然後停下來等人

圖 15

- 一步一步等，速度等於人的速度

圖 4

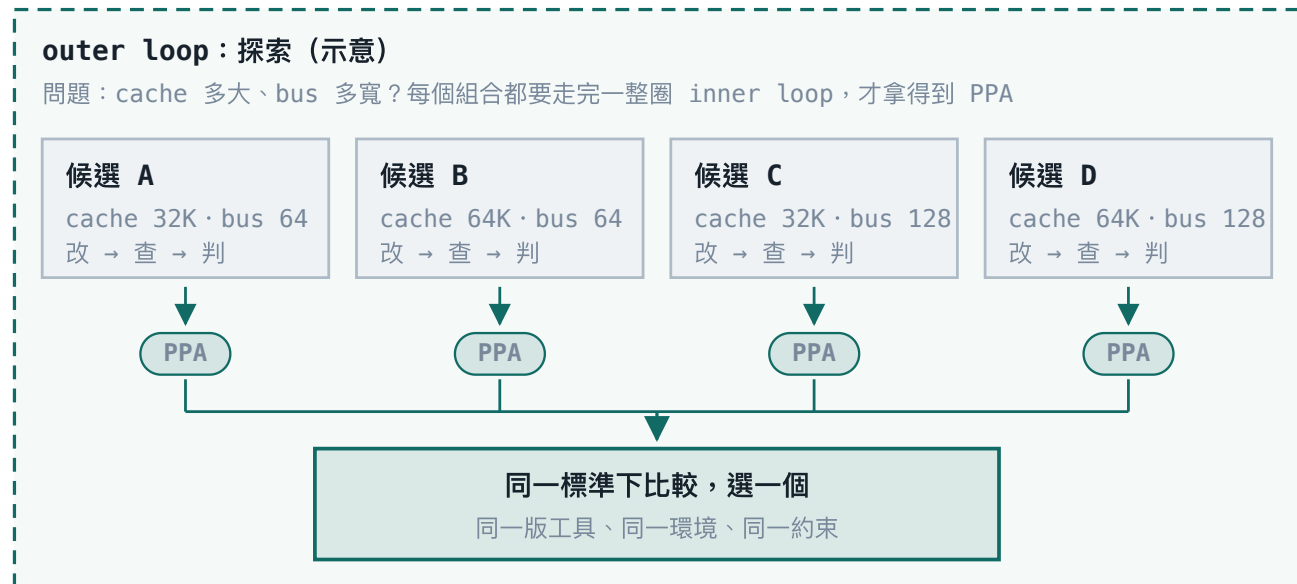
- 平行幾個 agent 也一樣，排同一條人龍

括號裡的圖號指《把版控當備份的團隊》

沒有 CI/CD，一圈裡四個步驟靠人：設環境、跑、看、判；迴圈轉多快由人的空檔決定。

兩圈之間環境和標準不一樣，連「這一圈比上一圈好」都說不準。

Outer loop：真正要比的是 N 個候選的 PPA，每個候選都是一整圈 inner loop



業界已經有機器在跑這個外圈

Synopsys 對 DS0.ai 的說法：

「autonomously search design spaces for optimal PPA solutions」

Cadence Cerebrus：自動探索 implementation flow，多 block、多人

前提（推論，非 vendor 所述）：

flow 能被機器帶參數呼叫

PPA 能被機器讀回來

每次跑的環境一致

出處：[research/loops-and-ai-multiplier.md](#)

要比較 N 個候選，每個候選都要走完一整圈 inner loop，拿到同一標準下的 PPA。

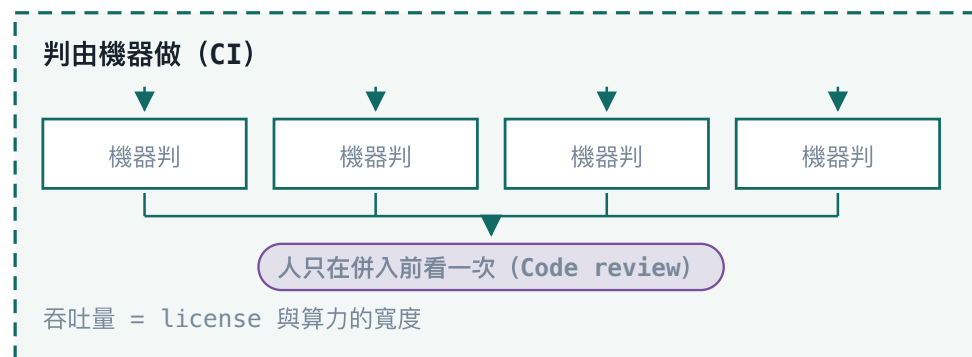
inner loop 轉不起來，outer loop 就沒有東西可以比；買了 DS0.ai 這類工具也接不上。

Agentic AI：便宜的是「改」，平行 N 個 agent 之後，瓶頸移到「判」

N 個 agent 平行做「改」：產生候選、改 RTL、寫 testbench



然後每一個都要被「判」



管理平行 coding agent 的工具 Superset 在部落格寫的：「every agent needs a human to review its code... it's the humans that don't scale」
另一位實作者的結論：產生與驗證可以平行，語意上的接受要串行。出處見 [research/loops-and-ai-multiplier.md](#)

agent 讓「改」變得很便宜，平行灑 N 個之後，瓶頸整個移到「判」。

判由人做，N 個 agent 排成一條人龍；判由機器做，人只在併入前看一次。

上限：平行的寬度由 license 與算力決定，比較要在同一環境下才算數

平行的寬度

由 license 與算力決定

- N 個 agent 要 N 份 license
- agent 再多也排隊
- 寬度是 CAD 與預算的事

一圈的底線

heavy flow 一圈仍然慢

- regression、synthesis 一圈還是那麼久
- AI 省的是等人的時間
- 省不了工具跑的時間

比較的前提

同一版工具、環境、約束

- 少一樣，PPA 的差異
- 就分不出是候選的差
- 還是環境的差（圖 14）

AI 的加成 = 不用人顧的圈數 × 可信的比較

把 AI 的效益講成「sim 變快」是講錯了；它加的是圈數和比較，而且三個上限都還在

CI/CD 直接解的是第三個（比較的前提）；前兩個是排程與預算的問題，但要先有 CI/CD 才排得起來。

平行有上限，一圈有底線，比較有前提；三個都是工程問題，講清楚效益才算得出來。

CI/CD 解的是比較的前提，也讓前兩個能被排程。

連鎖：沒有 CI/CD，inner loop 要人顧，outer loop 轉不完，倍數歸一



同樣 N 個 agent、同樣的 license，一段時間內轉完的圈數（長條是示意）

有 CI/CD



圈數多，而且可比

沒有 CI/CD



等於一個人的速度，而且圈與圈不可比

《把版控當備份的團隊》的症狀，各打斷鏈上哪一環

圖 2、3

環境散在 depot 外 → 每圈要人設環境

圖 14

兩台機器結果不同 → 圈與圈不可比

圖 15

狀態靠人填 Excel → 判靠人

圖 5

壞了很久才發現 → 圈沒有閉合

圖 8

main 隨時會壞 → agent sync 到壞的

沒有 CI/CD，先是 inner loop 轉不動，outer loop 自然沒得轉；AI 加速的只有「改」，倍數變回一。

《把版控當備份的團隊》裡的每個症狀，都在這條鏈上打斷一環。

對應：六個原則各自撐住迴圈的一個條件

原則	撐住迴圈的哪個條件	少了會怎樣
Small batches	一圈一件事	判出來的差異才歸得到某個改動
Single Source of Truth	每圈的條件一樣；agent 進得去就能跑	少了：兩圈不可比，agent 要人設環境
Traceability	每個結果連得回候選與環境	少了：比較表是假的
Continuous Integration	查和判由機器做	少了：每圈要人
Self-documenting	agent 不用人帶就看得懂	少了：每個 workspace 要人寫說明
Code review	人判放在併入前，一次做完	少了：人判散在每一圈，或完全沒有

六個原則的用處只有一件事：讓迴圈自己轉、能平行、能比較。
這就是 CI/CD 對這個團隊的意義，也是 AI 的倍數成立的前提。